

BayesR3 and PredictBayesR3 user manual

Contents

BayesR3 and PredictBayesR3 user manual.....	1
BayesR3	2
The parameter file.....	2
Command line parameters	3
Version identification	9
Input files	9
Phenotype file	11
Genotype file.....	12
Split genotype files: genotype file prefix.....	13
Pedigree file	14
GRM (A-matrix) file	15
Reference file.....	16
Validation file	16
Fixed effects file.....	16
Classification file	19
Output files	19
Output file names	20
The Effects file.....	20
The Model file.....	21
Classification distributions	23
The Poly file	23
The Log file	24
The SNPtrace file	24
The yhat file	24
PredictBayesR3	24
Validation:.....	25
Correlation:	25
Prediction:	26
Appendix	26
Common errors encountered:	26
Example run time output	26
Example slurm script.....	28
Multi-trait PCA values	30
Change log.....	31

BayesR3

BayesR3 takes a minimum of one command line argument, **<parameter_file>**:

```
%> bayesR3 <parameter_file>
```

The parameter file

The parameter file argument gives the name of a file used to pass a set of run time parameters as shown in Figure 1. The format uses a name (id) value style of input:

```
-<command id> <command value> [optional command comment]
```

Note the '**minus**' character in front of the **command id**. In the parameter file there is just one parameter per line specified and the line order of the parameters is not important. The parameter file can also contain blank lines and comment lines. A blank line is a line that only contains a carriage return, while a comment line is a line that begins with double forward slash '/' as shown in Figure 1. All lines beginning with '/' or are blank are simply skipped.

Command lines are identified by the lines beginning with a minus sign. After this the command identifier name is specified, which is then followed by its value(s). Some commands can take 1 or more values and for such commands, a comma separated list of values is expected. **Note in a comma separated list of values no spaces are expected, all values are on one line and there is no limit to the number of values allowed.** For example:

```
-pi 0.94,0.049,0.01,0.001
```

Notice also in Figure 1 that individual commands can also contain comments. However, command comments must be all on one line following its command. If they go across lines each line must begin with //. Also note that in this example, Figure 1, each command comment starts with a double forward slash (/). However, on a command line the double slashes are optional. All that is really required is a space separating the command value from the command comment. A detailed explanation of each command is given in Table 1.

```
//----- INPUT FILE information
-genf ../jerBull_genotype_BayesR.txt.gz //Name of genotype file (can be a '.gz' file and or split prefix)
-parts 0 // if 1 interpret genotype file name as a split file prefix. Default is zero
-phef ../jerBull_pcaNANs_FMP_BayesR.txt //Name of phenotypefile (can be a '.gz' file)
-ped NA //Optional name of pedigree file if available (can be a '.gz' file)
-grm NA // genomic relationship matrix GRM
-fixf NA //Optional name for a fixedeffects design file (can be a '.gz' file)
-ref ../jerBull_validation.txt //Optional file name giving a set of reference IDs (can be a '.gz' file)
-valid NA // validation file used by predictBayesR3
-classf NA //Optional file name containing a classification for each SNP, to run a BayesRC analysis
-sc 1 //Scale and centre genotypes. If 0 use raw genotypes values 0,1 or 2
-tc 1 //Column number(s) of the phenotype(s) to analyze
-wtc NA //Weight columns in phenotype file to use
-h 0.5 //Trait heritability(s)
-maf 0.002 //Minimum allele frequency

//----- OUTPUT FILE information
-path ../keep //Path directory to store output files. Will be created if needed
-pre testing //Output file prefix
-d 1 //Output detail; 1 = means only, 2 = full output
-fext csv //Output file extension type, one of {csv, txt, tab}
-chains 1 //A chain represents a computer thread, minimum value = 1

//----- MCMC parameters
-l 50000 //Number of iterations
-burn NA 25000 //Length of burn in. If NA then half the iteration number is used
-bs NA 12 //Block size. If NA then sqrt of number of records is used
-thin 1 //Thinning rate on output values. Minimum value is 1. Only applicable if bs = 0
-drop NA // drop rate. For example -drop 200 → drop a SNP if in the last 200 iterations it was never in the model.
-plusAlpha NA // if 1 add dropped SNPs to alpha distribution

//----- Mixture Distribution parameters
-pi 0.94,0.049,0.01,0.001 //Proportion in each SNP effect distribution
-alloc 0.0,0.0001,0.001,0.01 //Allocation of additive genetic variance
-alpha 1,1,1,1 //Dirichlet alpha parameters

//----- Optional extras
-s NA //Optional Seed value(s).
-DAV 1 //if 1 then determine additive genetic variance
-timing NA //comma separated sequence of integer values
-force 0 //If 1, then for a multi-trait analysis all SNPs will be associated to all traits
-show 0 //Used to check and show the input parameters.
-L 0 // if 1 redirect all screen output to a log file. Default is zero
```

Figure 1: Example parameter file. Notice comments after each parameter. In this example each comment starts with a double forward slash (/). A comment on each line is optional however the double slashes is just a matter of choice. All that is required is a space followed by some text after each parameter value.

Command line parameters

BayesR3 also takes separate command line arguments. Command line options will override the parameters specified in the parameter file. Command line options, follow the same rules as applied to the parameter file other than command line arguments are not expected to contain comments:

```
Usage: bayesR3 <parameterFile> [optional parameters]
       <parameterFile> The name of a file containing a list of parameters
Optional parameters:
-L 0 if 1 then redirect screen output to a logfile is off. Default is zero
-s <seed> random number seed(s). Default is as a seed_seq of 61 integers generated C++'s random_device
-genf <filename> name of genotype file
-parts <0 or 1 or NA>. If 1 consider genotype file name as a file prefix
```

```

-phef <filename> name of phenotype file
-grm <filename> name of GRM file
-fixf <filename> name of fixed effects file
-classf <filename> name of SNP classification file
-ref <filename> name of Reference set file
-valid <filename> name of Validation set file
-path <path> directory path for output files
-pre <value> output file prefix
-d <value> output detail; 1 = means only, 2 = full output, 3 collect SNP variances
-ped <file name> name of pedigree file
-I <value> number of iterations
-burn <value> number of iterations for burn in
-bs <value> block size must be a positive integer number
-thin <value> thinning rate
-pi <comma values> mixture model proportions eg: 0.95,0.04,0.02,0.01
-alloc <comma values> mixture model allocations eg: 0.0,0.01,0.1,1
-alpha <comma values> mixture model Dirichlet priors
-h <comma values> trait heritability (h^2) values. One value for each trait to analyse
-maf <value> //Minimum allele frequency
-wtc <comma values> trait weight columns in the phenotype file
-sc <value> scale and centre genotypes. Zero represents no scaling while 1 represents scaling
-tc <comma values> trait column(s) to analyse
-fext <type> output file extension types csv = comma separated, txt = space separated
-DAV <0 or 1> if 1 determine additive genetic variance from data with flat priors
    if 0 estimate it from the traits and heritability
-force <0 or 1> if 1, then for multi-traits all SNP are considered to be associated
-show Check and show input parameters and exit without running the analysis
-chains <value> an integer value greater than or equal to 1
-timing <comma values> run time analysis
-drop <value> or NA. A positive integer value, used to control the dropping rate of SNPs. NA specifies
no dropping
-plusAlpha <0 or 1 or NA>. Specifies to add count of dropped SNPs to the alpha distribution

```

Detailed specification for these command options are given in Table 1.

Table 1: Explanation of command line arguments.

Parameter	Description
-L	The -L flag specifies that all standard output (that which usually gets printed on the screen) is saved to a log file. The suffix 'log' is appended to the output filename prefix, stored in the output path. This file can be viewed "live" using the tail command under Linux: <pre>tail -f -n +1 <output_path>/test.log.txt</pre> or via the powershell under windows: <pre>Get-Content -Path "output_path\test.log.txt" -Wait</pre> To exit tail or Get-Content, at the command line, press the <ctrl>c keys simultaneously, where <ctrl> represents the control key.
-genf	Used to specify the genotype filename. Filename can include a directory path and the file can be a .gz file. A filename can also be a split prefix name, this prefix should only be part of the split subpart files. See: Split genotype files: genotype file prefix
-parts	If 1 then treat the genotype file names as a prefix name, otherwise the genotype file name represents a single file. Default value is 0
-phef	Used to specify the phenotype filename. Filename can include a directory path and the file can be a .gz file.
-ped	An optional pedigree file that contains three columns: animal ID, sire ID, dam ID in this order. Default is NA. This file can be a .gz file. The IDs can be either numeric or alphanumeric. Zero is used to represent a missing sire or dam. It is desirable that the pedigree file should also contain the non-genotyped ancestors of the genotyped animals as these ancestors provide

	links between families.
-grm	An optional genetic relationship matrix file, which contains a $n \times n$ matrix of record relationships as determined by their SNPs. Or it can be an A-matrix as determined from a pedigree. The value of n must be at least as large as the number of phenotypes to be analyzed or larger. This file is expected to contain $n+1$ columns. Where the first column gives the record IDs. There is no specified order of the records required. Default is NA. This file can be a .gz file. The IDs can be either numeric or alphanumeric and must match to those specified in the phenotype, genotype and fixed effects file.
-fixf	Used to specify the fixed effects filename. Filename can include a directory path and the file can be a .gz file.
-ref	Name of a file containing a list of reference ID's. The animals/records specified in the reference set are used in the analysis. All other animals/records will be ignored. This file can be a .gz file.
-valid	Name of a file containing a list of validation ID's. The animals/records specified in the validation set are used only in a predictBayesR3 analysis. All other animals/records will be ignored. This file can be a .gz file.
-classf	Name of a file containing a classification for each SNP. This file can be a .gz file.
-path	The path option is used to specify a directory where the output result files will be saved to. If the given path doesn't exist, then it will be created. If the path contains a directory tree, then all directories along that path if needed will also be created.
--ppath	A directory path used by predictBayesR3 to store its results if set. Default is NA and in which case predictBayesR3 uses the directory defined by -path .
-pre	The prefix option is used to specify a leading common name for the output results files. Therefore, all result files will have the same starting prefix. The prefix can also contain a leading directory path. However, if this path doesn't exist it won't be created, and an error will be generated.
-s	The -s flag specifies an optional seed for the random number generator. As all implemented statistical distributions used by bayesR3, such as the Dirichlet, Beta, Chi-squared, uniform and normal distributions, are deterministic, they have been configured to use the same random number generator engine to ensure that repeat runs with the same seed(s) on the same data will generate 100% the same output result sets. For a multi-chain analysis, as only a single set of seed values can be specified therefore, the specified seed values will be divided into equal (if possible) seed sets for each chain. For multi chain analysis the number of seeds must be equal or greater than the number of chains. If a seed is not specified, then a non-deterministic set of 61 seeds are used to seed the random number generator engine at program startup and which will be printed to screen or saved to the log file at run time. Therefore, a seed can be a single value, or a comma separated sequence of seed values such as: bayesR3 <parameter_file> -s 101,1193,4283 NB: No spaces are allowed between the seed values.
-d	The -d option is used to specify the level of output detail. If d is set to 1 then only the mean values are specified and only the mean SNP effects are collected and saved in the effects file. If d is set to 2 then, an extra file is created, and a trace of all the SNP effects per post-burn iteration is saved. The suffix "_SNPtrace" is appended to the file name for these results.

	NB: the trace only includes the post burn in iteration results. To get every result at every iteration the burn in needs to be set to zero. The trace file will have (iterations - burn)/block_size rows by the #SNP columns. The order of the SNPs will be the same as in the genotype file.
-I	The number of iterations to perform.
-burn	The number of iterations to ignore. If this parameter isn't set or set to NA, then it will automatically be set to half the number of iterations. Default value NA
-chains	A chain represents a computer thread. Each chain also represents a replicated run. Therefore, specifying 3 chains implies that 3 sets of output files will be produced with file name suffices chn1, chn2, and chn3 being adding to each file in each output set. Default value 1
-bs	Optional value. If set to NA or -1 (default value) then block size is determined From $\sqrt{\#Records}$, where #Records represents the number of records being analyzed. If block size is set to zero, then a non-blocked Bayes R will be run. For a blocked bayesR, the block size (ideally, should be a multiple division of the #SNPs). Determining the best block size can be a little difficult, but as a rule $bs = \sqrt{\#Records}$ if block size if left unspecified that is: -bs NA. The following R function (findMod) can at times help: <pre>findMod = function(nsnps, range=1:100) { df = data.frame(); for(i in range) { d = data.frame(i=i, mod=nsnps%%i, diffp= min(nsnps%%i, abs(i-(nsnps%%i))/i)) df = rbind(df, d) } df = df[order(df\$diffp),] df }</pre> The parameters passed to findMod are the number of SNPs (nsnps) and the range of values (block sizes) to consider: <pre>> head(findMod(856333, 1:100)) i mod diffp 1 1 0 0.00000000 92 92 89 0.03260870 52 52 49 0.05769231 46 46 43 0.06521739 94 94 87 0.07446809 87 87 79 0.09195402</pre> The above output shows that 856333 has no divisor, other than 1, where the quotient is a whole number. Therefore, for this example, we want to pick the divisor, which gives the largest remainder. In this case it is 92. Otherwise pick the largest divisor which gives no remainder. N.B: The maximum block size should be no more than the square root of the number of records in the analysis.
-pi	Mixture proportions. A comma separated list, where the number of values defines the number of mixture components and each value defines the starting value for each component (the first value is reserved for the zero component); For example:

	0.95,0.03,0.01,0.01. Setting this value sets nd to equal the number of items in the list.												
-alloc	Mixture allocations. List of values separated by commas, each representing the scaling factor for the variance of a mixture component. Note that the number of values should match that in -pi. Setting this value sets nd to equal the number of items in the list.												
-alpha	Mixture Dirichlet priors. List of values separated by commas. Note that the number of values should match the number of mixture proportions. Setting this value sets nd to equal the number of items in the list.												
-h	Trait heritabilities. A comma separated list of values. Each heritability value is a value between 0 and 1. The order of these values are expected to match the order of the phenotype columns to be analyzed. That is, the first heritability corresponds to the first trait specified etc. If the number of heritabilities = 1, and the number of traits is greater than 1, then the given heritability will be used for all traits.												
-tc	trait column(s) in the phenotype file to analyze. A comma separated sequence of values from 1 to nc, where nc is the number of trait columns in the phenotype file. 1 is the first column after the animal id column in the phenotype file for example: bayesR3 <parameter_file> -tc 5,6,7,21 specifies that columns 5, 6, 7 and 21 in the phenotype file are to be used for analysis. When more than one column is specified a bayesR3 multi-trait analysis is performed. Note: the number of traits to be analyzed must match the number of heritability values, unless the number of heritabilities is 1. In this case, the one heritability value specified is used for all traits.												
-wtc	Weight column(s) in the phenotype file. A comma separated sequence of values from 1 to n_w , where n_w is the number of weight columns in the phenotype file. 1 represents the first column after the record id column. NA is used to specify no weight columns are specified. For multi-trait analysis, the wtc can specify just 1 column or a comma separated list of column numbers. If only one column is specified, then the weights in that column are used for all traits. If multiple columns are specified, then the number of weight columns specified must match the number of phenotype columns specified.												
-sc	Scale and centre genotype data. 0 represents no scaling while 1 represents scaling. The elements of V, the genotype matrix data, are often coded as either 0, 1 or 2 for respective homozygote first/major allele, heterozygote and homozygote second/minor allele. Equivalently, they can be coded in terms of allelic dosages. The elements of V are normalized via: $v_{ij} = \frac{v_{ij}^* - 2p_j}{\sqrt{2p_j(1 - p_j)}}$ Where v^* represents a coded allele, or an allelic dosage value, and $p_j = \sum_i v_{ij}^* / (2n_t)$ represents the frequency of the minor allele for SNP j, n_t represents the number of records and where i indexes across the records. Default value 1												
-fext	Specifies the output file extension types to be either csv or txt. Csv files are comma delimited, txt files are either space or tab delimited as summarized: <table><tr><th>Type</th><th>Delimiter</th><th>File extension</th></tr><tr><td>cvs</td><td>comma</td><td>.csv</td></tr><tr><td>txt</td><td>space</td><td>.txt</td></tr><tr><td>tab</td><td>tab</td><td>.txt</td></tr></table>	Type	Delimiter	File extension	cvs	comma	.csv	txt	space	.txt	tab	tab	.txt
Type	Delimiter	File extension											
cvs	comma	.csv											
txt	space	.txt											
tab	tab	.txt											

	For example: bayesR3 <parameter_file> -fext tab Specifies a tab delimited output style and will result in text files with file prefix 'txt'. Default value csv
-DAV	If 1, determine the additive genetic variance from the data. If 0, estimate the additive genetic variance for each trait using its variance and given heritability value. Default value 1
-force	If 1, then for multi-traits all SNPs are associated to all traits. For single trait analysis it has no effect. A forced multi-trait analysis is identical to a single trait analysis, for each trait analyzed. Default value 0
-show	Used to check and show the input parameters. BayesR3 analysis will not be performed. Default value 0
-timing	Only supported on single trait analysis. A comma separated sequence of integer values. The sequence of integers can contain 1 or more values, where each value specifies a block size, in any order. For example: bayesR3 <parameter_file> -timing 0,3,7,11,27,48 Will perform a timing analysis for your single trait analysis. This will output a 4 column by 6 lines text file, with format as described by the file extension. Each output line will contain 4 columns, where the first column gives the block size, and following 3 columns will contain 3 replicate timings for each of the block sizes specified. Default value NA
-drop	An optional integer value. NA, 0 or 0,0 are 3 ways to specify no dropping. The drop parameter is used to drop SNPs from analysis that have probabilities of being included in the model. Dropping only occurs during burn in. The drop parameter takes 1 or 2 comma separated values. Default value is NA, which is used to specify no dropping. For example: -drop 100 Specifies that during burn in, a SNP will be dropped from analysis if in the last 100 iterations it was not included in the model. In this case dropping can only occur on and after the 100th iteration. -drop 100,200 Specifies that during burn in, a SNP will be dropped from analysis if in the previous 100 iterations it was not included in the model. In this case dropping starts at the 200th iteration. -drop NA Specifies no dropping.
-plusAlpha	Used in conjunction with -drop and is an optional value, which is one of the following values: 0, 1, or NA. If 0 or NA is specified, then -plusAlpha is off. If 1, then -plusAlpha is on. The default state is off.

	Example usage: -plusAlpha 1 If on, then BayesR3's mixture counts for all dropped SNPs are added to the n Dirichlet alpha distribution, where n, usually set to 4, represents the number of mixtures in BayesR3's Gaussian mixture model. BayesR3 does this by keeping track of the number of times each SNP falls into the 1st, 2nd, ..., nth distribution. Then, when a SNP is dropped, its distributional counts are normalized to sum to one and these normalized values get added to the corresponding Dirichlet prior alpha distribution values. The net effect is that SNPs get dropped at a faster rate than when plusAlpha is off, and more SNPs will generally get pushed into higher distributional positions than when plusAlpha is off. This also means that some SNPs will get assigned larger effect values, which in turn can improve the prediction bias, but only if it's below 1 when no dropping is specified. Therefore, when plusAlpha is on the actual bias value is expected to increase a few percentage points. NB bayesR3 will issue a warning if -plusAlpha is on when dropping is off.
--	--

Version identification

BayesR3 uses the build date as its version identification. The build date is incorporated into bayesR3 prior to its release. The simplest way to get to the version identification is to call bayesR3 from the command line with no arguments and its build date will be displayed in the first line of the output:

```
> bayesR3
BUILD: Mar 16 2020 15:26:35
Usage: bayesR3 <parameterFile> [optional parameters]
      <parameterFile> The name of a file containing a list of parameters
....
```

The version identification is also included in the runtime output from bayesR3 and if the -L option is used and is set to 1 it will be saved in the log file for future reference. Date stamps, the file creation time associated with the bayesR3 result output files can also be used, indirectly, to identify the release version of bayesR3 responsible for their creation.

The version identification can be used to check consistency between runs. It is possible upon request to get previous versions of the bayesR3 executable if needed, and if you know the date of when you ran your analysis.

Input files

All input files read by bayesR3 are space delimited plain ascii text files; Also, all input files other than the parameter file can be a **‘.gz’** compressed file.

As genotype files can be very large (100s of gigabytes or terabytes), it is worthwhile in terms of disk storage to have these files saved in compressed form. The left panel in Figure 2 shows an example of the compression ratio achieved with gzip on a typical but small genotype file, 716 records by 37008 SNPs. The right figure shows the time required to read these files line by line using either C++’s ifstream or by igzstream (a C++ interface to zlib).

To create a **gzip** file under windows: browse to the location where the **file** you want to compress is located. Right click on the text file name and select **7-Zip > Add to archive...** From the pop-up window set the **Archive** format to **gzip** and then click the OK button to start compressing.

To create a gzip file under Linux use gzip. Example: **\$> gzip genofile.txt**, this will result in the **genofile.txt** file being replaced by **genofile.gz**.

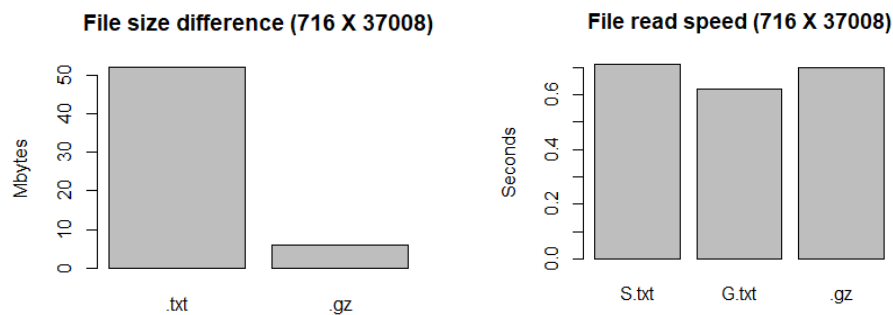


Figure 2: left panel gives the size of a .txt formatted genotype file to its compressed version (.gz). The right panel gives the read speed of these files, where S.txt is the read time of the .txt file using C++ ifstream, the G.txt result is for reading the .txt file using igzstream and the .gz result is from the .gz file being read using igzstream.

The order of reading of the various files are:

1. Pedigree file
2. Reference set or Validation set
3. Phenotype file
4. Genotype file
5. Fixed effects file
6. Classification file

If there is a reference set specified for analysis, then the IDs in this file will determine which records are read from the phenotype file, genotype file and fixed-effects file. If there is no reference set provided, then the IDs in the phenotype file determines which records from the genotype and fixed-effects files are to be included in the analysis. The order of rows in each of these files

don't need to be the same. BayesR3 will align them automatically using the ID's given separately in each file.

If a pedigree file is specified, then the pedigree file is handled differently. All pedigree information is read in to create a relationship matrix. After which, it is reduced in size, prior to analysis, to only include the records as determined by the reference file or by the phenotype file.

The validation file is used only by predictBayesR3, see below, and is ignored by bayesR3.

Phenotype file

A phenotype file (see Table 2) is a space delimited text file (txt), with no header line. All the phenotype data for a record should be presented on one line. The file should contain a minimum of at least two columns; The first column (column 0) should contain the record IDs. Subsequent columns should contain the phenotypes (traits) for each record, and any corresponding weights.

Individual phenotype (trait) values for any record can be missing. Missing values should be denoted by a '*', a '.' or by the character string 'NA'. Internally, missing values are handled by setting a zero weight for that missing value otherwise a weight of 1 is used for all non-missing values.

BayesR3 doesn't expect or specify any ordering of the trait or weight columns in the phenotype file. Other than they must appear after the ID column, which must be the first column in the file. The user specifies, via the parameter file or via the command line, which columns in the phenotype file are to be considered as the trait columns for analysis, and which columns, if any, are the weight columns. It's not an error to specify the same trait column multiple times. The phenotype file can contain more trait and weight columns than those specified for analysis.

With respect to the weight columns and for multi-trait analysis, weights can be specified in the parameter file by either:

1. NA in which case there are no user specified weights to include
2. Specifying a single value and in this case the weights in this column will be used as the weights for each trait to be analyzed
3. Specifying a comma list of weight columns. The order and the number of the given weight columns are expected to coincide with the order and number of traits given for the trait columns to analyze; that is, the first weight column will be associated to the first trait column specified in the parameter file and so on.

*Table 2: Example phenotype file. Notice there are 9 columns in this example, where the first column (column 0) gives the record IDs. The next 8 columns contain the trait and weight columns. In this example columns 1, 3, 5 and 7 are trait columns. While columns 2, 4, 6 and 8 are the corresponding weight columns. **Note also, that***

missing trait-values can be represented by an asterisk, *, NA or a full stop, ., as highlighted in the yellow table cells.

Gid109605	1.25	0.79	*	0.92	0.26	0.78	-1.16	0.79
Gid110228	0.64	0.69	*	0.68	0.16	0.68	-0.93	0.69
Gid110729	1.59	0.69	*	0.68	0.16	0.68	-1.65	0.69
Gid111552	0.55	0.69	-0.16	0.00	0.51	0.68	-0.33	0.69
Gid112124	1.09	0.62	0.13	0.64	0.28	0.59	-1.34	0.62
Gid112655	0.59	0.69	-0.88	0.68	.	0.68	-0.55	0.69
Gid113302	NA	0.7	0.34	0.68	.	0.68	-1.04	0.79
Gid113330	0.35	0.75	-0.17	0.68	.	0.68	-0.09	0.79

Genotype file

This file is also a space delimited text file with no header line. All the genotype data for a record should be presented on one line. The line (row) order of the genotype file does not need to match the order given in the phenotype file. Only those lines corresponding to records specified in the phenotype file or via a reference file (see below) will be extracted and used in the analysis. **All phenotype records to be analyzed need to match uniquely to one genotype record.**

The genotype file should contain at least two columns; the first column should contain the record ID. A genotype file containing **n** SNPs would therefore, contain (n+1) columns.

Missing genotypes are designate in the genotype file by the same codes as used for the phenotype file. That is a period(.), an asterix (*) or NA. Missing genotypes will be replaced by the mean genotype value for the given marker. If a SNP column is completely full of missing values, it's values will be set to 0 and it will get flagged as a monomorphic SNP, and hence ignored during further analysis.

1. The genotype for each SNP is entered as a code where 0 = 1 1, 1 = 1 2 or 2 1 and 2 = 2 2. In this case 0, 1 or 2 are the only values expected for each genotype value in each SNP column.
2. Genotypes can also be represented in terms of allelic dosages as shown in Table 3.

Table 3: Example of an allelic dosage genotype file.

Gid109605	1	1	0	0	1.962	2	0
Gid110228	1	0	2	0	1.944	0.017	1.997
Gid110729	1	0	2	1.916	0.023	1	0.91
Gid111552	1	0.008	1.994	1	0.999	0.168	1.995
Gid112124	1	0.967	1.997	0.127	1	1.986	1.963
Gid112655	1	0.008	1.999	0	1	1.976	1
Gid113302	1	0	2	1.996	2	0.373	1.919
Gid113330	1	1.983	1.995	1.993	1.968	0	2
Gid113744	1	0.867	1.998	0.02	1.79	0.389	0.603
Gid113999	1	0.001	1	1.987	1.529	0.003	1.999
Gid114084	1	1.995	1.999	1	1.752	1.999	0.206
Gid114169	1	1.998	1.999	0	1.969	1	2

Dosage values are derived from some linear transformation of the posterior genotype probabilities usually coming from an imputation step. For example:

$$0 * A/A + 1 * A/B + 2 * B/B = 0.4 + 2 * 0.5 = 1.4$$

Where the probabilities of the genotypes: A/A, A/B and B/B are 0.1, 0.4 and 0.5 respectively.

Split genotype files: genotype file prefix

As genotype files can be very large requiring bayesR3 to spend many hours just to load a terabyte genotype file, bayesR3 has the feature that it can read genotype files in parallel, thereby greatly speeding up the read time required. However, the file must be split into small parts (see below) and the **-parts** command must be turned on.

If the **-parts** command is on then just pass to bayesR3 via the **-genf** command the output prefix used to split the original genotype file into parts, using the UNIX split command (or equivalent, such as with Windows power-shell):

zcat subSet4e4.txt.gz | split - -l10000 subParts4e4

split options used:

- get input from stdin
- l break up by line
- subparts4x4 file prefix name

For a genotype file containing 65,000 records/lines the above linux split command will generate the following 7 files:

**subParts4e4aa subParts4e4ab subParts4e4ac subParts4e4ad
subParts4e4ae subParts4e4af subParts4e4ag**

Each containing 10,000 records/lines except for the last file (subParts4e4ag), which will contain only 5,000 records/lines. In this case the filename is passed to bayesR3 is:

-genf ./pathToFile/subParts4e4

For example:

Run with original file:

```
Reading genotype(s) from ../../Iona/Geno/subSet4e4.txt
Time to load genotypes: 171.113(sec)
genotype (rows,cols) = 41925,40000
```

Run using split sub parts:

```
Reading genotype(s) from ../../Iona/Geno/subParts4e4aa
Reading genotype(s) from ../../Iona/Geno/subParts4e4ag
```

```

Reading genotype(s) from ../../Iona/Geno/subParts4e4ad
Reading genotype(s) from ../../Iona/Geno/subParts4e4ae
Reading genotype(s) from ../../Iona/Geno/subParts4e4ab
Reading genotype(s) from ../../Iona/Geno/subParts4e4ac
Reading genotype(s) from ../../Iona/Geno/subParts4e4af
Time to load genotypes: 49.0261(sec)
genotype (rows,cols) = 41925,40000

```

This equates to 3.5 times speed up. The greater the number of cores and split parts the faster the read will be. **Also note the order of the file reading doesn't matter.**

To split and zip the genotype file in one step use:

```
zcat subSet4e4.txt.gz | split - -l10000 --filter='gzip > $FILE.gz' subParts4e4
```

An equivalent PowerShell command for file splitting by lines is:

```
$i=0; Get-Content subSet4e4.txt -ReadCount 10000 | %{ $i++; $_ | Out-File sub4e4.$i }
```

Note:

- **do not split a .gz file. Ungzip it first.**
- **the prefix should not be part of any other file name in the directory holding the split subpart files and or genotype file.**

Pedigree file

This file is also a space delimited text file, where the first three columns (see Table 4); are expected to represent animal ID, sire ID and dam ID and in that order. However, if the pedigree file has more than 3 columns only the first 3 columns are read.

Zero is used to represent a missing sire or dam. Therefore, it follows that no animal can have an ID of zero. The order of the rows is expected to be such that oldest animals/organisms precede the youngest; that is, parents come before offspring.

To keep the pedigree file as small as possible, the pedigree file should contain only those animals specified in the phenotype file and their relatives. This can be achieved via Phil's **distilPedigree** program.

Table 4: Pedigree file layout

JERAUSF000594196470	0	0
JERAUSF000594107778	JERNZLM000000064399	0
JERUSAM0000000633286	JERUSAM0000000604853	0
JERNZLM000000080404	JERNZLM000000071451	JERNZLF000JR951578F
JERNZLM000000064595	JERNZLM000000064399	JERNZLF000JR737891F

The population described in the pedigree can include IDs with rec-ords to be analysed or IDs without records for analysis and therefore, these animals get excluded. This is done by constructing an ‘A-matrix’. The A-matrix can be conceptualised as 4 blocks:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

Theses 4 blocks (sub-matrices) specify the relationships between the excluded animals A_{11} and the included A_{22} (animals or plants to be analysed). BayesR3 extracts the sub-matrix A_{22} and then inverts it to produce A_{22}^{-1} , which is the inverse pedigree relationship matrix of the included records, and this is the matrix that gets included into the analysis.

GRM (A-matrix) file

A genomic relationship matrix (GRM) is essentially equivalent to the A-matrix as described above; therefore, bayesR3 allows for the input of a GRM or an A-matrix file via the use of the “-**grm**” command. A GRM file or an A-matrix file is expected to be a space delimited file without any header line. It is also expected to contain a $n \times n$ matrix of record relationships as determined from some set of SNPs, or from some pedigree. This GRM/A-matrix file is also expected to contain $n + 1$ columns, where the first column contains the record IDs as shown in Table 5.

The value of n must be at least as large as the number of phenotypes that will get included from the phenotype file for analysis. As bayesR3 does automated matching of records, the order of the rows (lines) doesn’t need to match the order given in the phenotype file. Therefore, there is no specified ordering of the GRM or A-matrix file other than the first column must contain the record IDs.

Table 5: Example of a GRM file, which contains only 8 records

ID0001	5.461	0.158	0.458	0.137	-0.001	-0.160	0.449	-0.281
ID0002	0.158	5.333	0.202	-0.103	-0.081	0.113	-0.192	-0.233
ID0003	0.458	0.202	5.404	0.069	0.411	-0.068	-0.356	-0.237
ID0004	0.137	-0.103	0.069	5.274	-0.262	-0.037	0.204	-0.174
ID0005	-0.001	-0.081	0.411	-0.262	5.414	-0.255	0.225	0.024
ID0006	-0.160	0.113	-0.068	-0.037	-0.255	5.161	-0.254	0.266
ID0007	0.449	-0.192	-0.356	0.204	0.225	-0.254	5.417	-0.182
ID0008	-0.281	-0.233	-0.237	-0.174	0.024	0.266	-0.182	5.336

Once read, the GRM will be treated as the A matrix as specified for the pedigree file. It will be condensed to A_{22} and inverted to produce A_{22}^{-1} .

The following R code gives an example of creating a GRM (G) from a BayesR3 genotype file. This is based on the formula (VanRaden 2008):

$$G = \frac{ZZ'}{2 \sum_i p_i(1 - p_i)}$$

Where Z is the genotype matrix, Z' is its transpose and p_i is the minor allele frequency for the i^{th} genotype in Z , Note the Division by $2\sum p(1-p)$ scales the GRM to be analogous to the numerator relationship matrix A .

```
crtGRM = function(genofile)
{
  # see: VanRaden PM. Efficient methods to compute genomic predictions.
  #           J Dairy Sci. 2008 Nov;91(11):4414-23.
  m = read.table(genofile, sep = ' ', header = F)
  ids = m[,1]
  colnames(m) = NULL
  m = data.matrix(m[, -1])
  nr = nrow(m)
  p = colSums(m) / (2 * nr)
  denom = 2*sum(p*(1-p))
  a = round((m %*% t(m)) / denom, 3)
  df = data.frame(cbind(ids, a))
  #write output GRM file
  require(tools)
  gf = paste(file_path_sans_ext(genofile), "GRM.", file_ext(genofile),
    sep = '')
  write.table(df, file = gf, col.names = F, row.names = F, sep = ' ')
  df
}
```

Figure 3: R code to create a GRM from a genotype file.

Reference file

This is an optional text file, and when specified it is expected to contain only the ID's for the records to include in the analysis. The file is expected to contain a single column of IDs; that is, one ID per line. However, if the file contains multiple columns of text, then they are expected to be separated by spaces but only the first column of text is used as the identifying reference set.

The reference file is used to specify which records are to be included in the current analysis of bayesR3.

Validation file

Validation files are used only in predictBayesR3, where if specified in the parameter file the validation file is used to specify which entries to be included for analysis by predictBayesR3. Its format is as specified for the reference file.

Fixed effects file

This file is also a space delimited text file (see Table 6). All the fixed effects data for a record should be presented on one line. This file is a **design matrix** allocating records to fixed effects. The first column contains the record IDs. Subsequent columns specify the fixed effects associated with each ID.

Table 6: An example fixed effect file. The first column contains the record IDs.

Jer148847	1	0	1	0	1	12.3
Hol120383	1	0	0	0	0	23.4
Hol116748	1	0	0	0	0	13.5
Aus174112	1	1	0	0	0	20.9

Aus116699	1	1	0	0	1	25.5
Jer112903	1	0	1	1	0	25.5
Hol119769	1	0	0	1	0	20.7
Aus182492	1	1	0	0	1	15.8
Aus161347	1	1	0	0	1	14.4
Jer186735	1	0	1	0	1	12.8

A design matrix can be constructed using R. For example, Table 7 shows some of the contents of an R data frame (df) relating record IDs to several effects (**Breed**, **Gender**, **Tr** and **Age**), which will be treated as fixed effects.

Table 7: An R data frame (df) containing fixed effects used to create a design matrix. Note that columns Breed, Gender and Tr are factors. Breed has 3 levels (Holstein, Jersey and AusRed), Gender has 2 levels (F and M) and Tr has two levels (1 and 0).

ID	Breed	Gender	Tr	Age
Jer148847	Jersey	F	1	12.3
Hol120383	Holstein	F	0	23.4
Hol116748	Holstein	F	0	13.5
Aus174112	AusRed	F	0	20.9
Aus116699	AusRed	F	1	25.5
Jer112903	Jersey	M	0	25.5
Hol119769	Holstein	M	0	20.7
Aus182492	AusRed	F	1	15.8
Aus161347	AusRed	F	1	14.4
Jer186735	Jersey	F	1	12.8

In Table 7, the **Breed**, **Gender** and **Tr** columns are considered by R as factor variables, while Age will be treated as a numerical covariate. A summary of the data frame gives:

```
> summary(df)
      ID      Breed  Gender Tr      Age
Aus101017: 1   AusRed  :7   F:13  0:13  Min.   : 7.50
Aus116699: 1   Holstein:9   M: 7   1: 7  1st Qu.:15.45
Aus126301: 1   Jersey  :4                      Median :20.80
Aus161347: 1                                Mean   :19.72
Aus174112: 1                                3rd Qu.:24.38
Aus175274: 1                                Max.   :26.20
(Other)   :14
```

From the above output it is seen that the largest **Breed** level is Holstein, which includes 9 animals. The largest **Gender** level is F, and the largest **Tr** level is 0. The order of levels shown above is the same as the order for those factors. What is needed is to have **the largest group level as the first level for each factor**.

The following section is an example R session used to convert the data frame df into a suitable design matrix (dm), as seen in Table 6. This code also shows how to save the design matrix, together with the IDs, as a space delimited file, which is given the name: FixedEffects.txt

```

> str(df)
'data.frame': 10 obs. of 5 variables:
 $ ID : chr "Jer148847" "Hol120383" "Hol116748" "Aus174112" ...
 $ Breed : chr "Jersey" "Holstein" "Holstein" "AusRed" ...
 $ Gender: chr "F" "F" "F" "F" ...
 $ Tr : int 1 0 0 0 1 0 0 1 1 1
 $ Age : num 12.3 23.4 13.5 20.9 25.5 25.5 20.7 15.8 14.4 12.8

> df[2:3] <- lapply(df[2:3], factor)
> str(df)
'data.frame': 10 obs. of 5 variables:
 $ ID : chr "Jer148847" "Hol120383" "Hol116748" "Aus174112" ...
 $ Breed : Factor w/ 3 levels "AusRed","Holstein",...: 3 2 2 1 1 3 2 1 1 3
 $ Gender: Factor w/ 2 levels "F","M": 1 1 1 1 1 2 2 1 1 1
 $ Tr : int 1 0 0 0 1 0 0 1 1 1
 $ Age : num 12.3 23.4 13.5 20.9 25.5 25.5 20.7 15.8 14.4 12.8> makeFixedEffects(df)

> print(makeFixedEffects(df, ../Pheno/FixedEffects.txt))

```

	ID (Intercept)	BreedHolstein	BreedJersey	GenderM	Tr1	Age
1	Jer148847	1	0	1	0	12.3
2	Hol120383	1	1	0	0	23.4
3	Hol116748	1	1	0	0	13.5
4	Aus174112	1	0	0	0	20.9
5	Aus116699	1	0	0	0	25.5
6	Jer112903	1	0	1	1	25.5
7	Hol119769	1	1	0	1	20.7
8	Aus182492	1	0	0	0	15.8
9	Aus161347	1	0	0	0	14.4
10	Jer186735	1	0	1	0	12.8

where the makeFixedEffects R function is:

```

makeFixedEffects = function(pheno, toFile=NULL)
{
  # Create a fixed effects file suitable for bayesR3
  # this function assumes that 'pheno' is a data frame containing
  # 1 or more factor columns of fixed effects.
  # Each factor column should contain 2 or more levels.
  # It is also assumed that column 1 of 'pheno' contains the record IDs.
  # The first column of the design matrix is for the mean
  # Returns a data frame containing the design matrix

  fix = data.frame(pheno[, -1]) ## strip off the ID column
  id = which(sapply(fix, is.factor)) # find factor columns
  df2 = NULL
  if (length(id)) {
    for (i in id) {
      t = table(fix[, i])
      fix[, i] = factor(fix[, i], levels = names(t)[order(t, decreasing = T)])
    }
    dm = model.matrix(~ ., fix)
    df2 = cbind(ID = pheno[, 1], as.data.frame(dm))
    if (!is.null(toFile))
      write.table(df2, file = toFile, row.names = F, col.names = F, sep = ' ', quote = F)
  }
  invisible(df2)
}

```

Note, any type of valid design matrix can be constructed, including interactions terms.

Note the fixed effects design matrix file is expected to be a space delimited text file that contains no header line and the 2nd column to be all 1's, which is used to determine the intercept (mean).

If a fixed effects file is specified, once it's read and its rows have been matched up according to the reference file or phenotype file, each column in the resulting fixed effects matrix, except for the intercept column, are checked for variance. Any columns where the minimum and maximum coefficients are equal will be automatically excluded from the analysis and the effects for these columns will be set to zero in the output.

Classification file

This is an optional space delimited file that contains no header line. This file allows SNPs to be classified into two or more groupings, where each classification group is assigned its own mixture model. The initial mixture model assigned to each group is defined as the mixture model specified in the parameter file, and or specified on the command line.

A classification file contains a list of the SNPs as specified in the genotype file together with their classification. The order (line-order) of SNPs in this file is the same as the order (column-order) in the genotype file. Therefore, the number SNPs in the classification must match the number of SNPs given in the genotype file. An example of some entries from a classification file is shown in Table 8.

The file must contain only four columns in the following order:

1. Alpha numeric SNP ID. This is just unique identify. Here it used to encode the chromosome and SNP position on that chromosome.
2. Chromosome number.
3. Base pair position within chromosome.
4. SNP classification. Classifications should be numeric, consecutive (i.e. no gaps between classifications) and numbered from 1 to the number of classifications.

Table 8: Example entries in a classification file

chr1.353745	1	353745	2
chr1.471078	1	471078	2
chr1.516404	1	516404	2
chr1.845494	1	845494	2
chr1.883895	1	883895	1
chr1.929617	1	929617	1
chr1.950841	1	950841	1

Output files

Output file names

BayesR3 can output up to 8 types of files (as summarised in Table 9) and the names of the output files are constructed in the following manner:

```
<path>/<filePrefix>_<suffix>[.chnX].<txt|csv>
```

where **<filePrefix>** is the output file prefix provided in the parameter file. The **suffix** is determined by bayesR3 and is used to assign and distinguish various output results as described in Table 9. The **.chnX** extension is only used when the number of chains specified by the user is greater than 1. The designation **X** represents a number from 1 to N, where N is the number of chains specified. Therefore, if 3 chains are specified then 3 sets of output files will be generated, one from each chain. Each chain set of files represents replicate runs, each seeded with a different set of nondeterministic random seed values.

Table 9: Output file descriptions summaries. See documentation for more detail information on each output file

Suffix	Description
Effects	Gives the SNP effects and posterior inclusion probabilities (discussed below)
Model	Contains posterior samples of the model parameters and the fixed effects (discussed below)
Poly	Contains the effects due to the pedigree
log	A log file of the runtime parameters, command line and standard output produced by bayesR3 (discussed below)
SNPtrace	File(s) containing a trace of all SNP effects (discussed below)
yhat	Gives the predicted y-values obtained from the model. It can be considered the average value of the response variable given the input matrices and the determined effects.
Timings	Gives a text file which contains a set of timing results, see Table 1.
aveEffects	From a multi-chain analysis. The SNP effects and their distribution proportions from each chain are collected averaged and stored in the aveEffects file. The SNP effects results will also be stored along with their variance across the chains.

The Effects file

The suffix **'_Effects'** is appended to the output. The effects file will have a row for each SNP and the SNP row order, on output, will be the same as SNP column order found in genotype input file (see Table 10). For monomorphic SNPs, although they are excluded from the analysis, in the Effects file their associated rows will contain all zeros.

This file is a user specified delimited file that contains the mean posterior estimates for each individual SNP. The column labelled **Freq** gives the frequency for the minor allele for each SNP. The column labelled **Beta** gives the SNP mean effect, and the posterior inclusion probabilities of the SNP effect in the mixture classes 1 to k, are identified by the column headers **PIP1, ..., PIPK**. For example, Table 10 shows that the first SNP, has a 71 % probability that its coefficient - 88.6885, falls in the fourth distribution (PIP4).

Table 10: Example_Effects output.

Freq	PIP1	PIP2	PIP3	PIP4	Beta
0.575	0.0235	0.0083	0.2595	0.7087	-88.6885
0.425	0.2799	0.089	0.4555	0.1756	32.8407
0.55	0.4856	0.0917	0.2934	0.1293	-22.9376
0.475	0.7311	0.1285	0.0649	0.0755	9.8867
0.4	0.53	0.1186	0.3239	0.0275	14.1388
0.5	...	...	...	...	...

For multi-trait analysis the effects file layout is slightly different, as shown in Table 11. It still has a row for each SNP effect, and the order is the same as the order given in the genotype file. However, it has a set of PIP and Beta columns for each trait analyzed, with the trait number appended to the column headers. The trait number reflects the order in which the trait columns were entered. For example, for trait columns specified as: “-tc 6,5”, trait 1 refers to trait column 6, and trait 2 is referring to trait column 5.

It also contains two Q columns, where Q1 gives the unassociated probability of the SNPs, while Q2 gives the associated probability for each SNP. If a SNP is deemed unassociated, it signifies that it has no effect on any of the traits analyzed for that SNP. These columns help in determining if a SNP is jointly contributing to the analyzed traits.

Table 11: Example_Effects file from a multi-trait analysis

Freq	Q1	Q2	PIP1.1	PIP2.1	PIP3.1	PIP4.1	Beta.1	PIP1.2	PIP2.2	PIP3.2	PIP4.2	Beta.2
0.575	0	1	1	0	0	0	0	0	0	0.09	0.91	3
0.575	0	1	0	0	0.22	0.78	4.00	1	0	0	0	0
0.425	0.44	0.56	1	0	0	0	0	1	0	0	0	0
0.55	0	1	0	0	0.09	0.91	5.00	1	0	0	0	0
0.475	0.48	0.52	1	0	0	0	0	1	0	0	0	0
0.4	0.48	0.52	1	0	0	0	0	1	0	0	0	0
0.5	0.45	0.55	1	0	0	0	0	1	0	0	0	0
0.55	0	1	0	0.03	0.41	0.56	-2.00	1	0	0	0	0
0.75	0.54	0.46	1	0	0	0	0	1	0	0	0	0
0.6	0	1	1	0	0	0	0	0	0	0.06	0.94	-3

The Model file

The suffix ‘_Model’ is appended to the output prefix<id>. This file will contain a column for each trait analyzed in the order of the trait-columns specified in the parameter file or via the command line. This file is also used to hold the classification class proportions for each group specified in the classification file.

Each column other than the first and those columns associated with the last N-lines of this file, which are used to hold the grouping classification proportions (see below), and N is the number of classification groups, will contain:

1. **R²**, which gives the goodness of fit for the model to the data analyzed
2. **Mean**, the model intercept
3. **V_p**, the trait/phenotype variance of the input data
4. **V_a** the genetic variance explained by the SNPs
5. **V_e** the residual variance
6. **V_g** the variance of the genetic effects explained by the pedigree (if specified)
7. **V_{yhat}** the variance of the y-hats.
8. **nMono** the number of monomorphic SNPs detected, as specified according to the minimum allele frequency given in the parameter file
9. **Associated** gives the number of SNPs considered to be associated with the model. For single trait analysis this will include all the SNPs minus the monomorphic SNPs
10. **nSNPs** the number of SNPs considered to be in the model for that trait
11. **Nk₁, ..., Nk_k**, gives the number of SNPs detected to be in the mixture components 1 to k
12. **Pk₁, ..., Pk_k**, the proportion of the SNPs in the mixture components 1 to k
13. **Vk₁, ..., Vk_k** are the sum of squares of the SNP effects in the mixture components 1 to k
14. **F₁, ..., F_n** gives the fixed effects, if any; where, F₁ will give the fixed effect for the 1st effect in the fixed effects file, F₂ the 2nd and so on. **Note F₁ if present will be a replication of the Mean row**
15. **VF₁, ..., VF_n**, gives the variances associated with each fixed effect.

Table 12: Example _Model file output:

	Trait 1	Trait 2
R ²	0.5775	0.4879
Mean	8.554	23.11
V _p	195.8	319.8
V _a	66.4	129.3
V _e	95.76	189.4
V _g	0	0
nMono	0	
Associated	977.591	
nSNPs	944.1	949.3
Nk ₁	33.44	28.3
Nk ₂	938	944
Nk ₃	5.57	4.758
Nk ₄	0.54	0.5384
Pk ₁	0.03421	0.02895
Pk ₂	0.9595	0.9656
Pk ₃	0.005698	0.004867
Pk ₄	0.000552	0.000551
Vk ₁	0	0
Vk ₂	63.15	123.7
Vk ₃	2.629	4.239
Vk ₄	0.433	0.8711
F ₁	8.554	23.11
F ₂	-0.1268	-0.1428
F ₃	0.1322	0.2123
F ₄	0.3285	0.1172

F5	-0.04526	-0.3771
F6	-0.2516	-0.3463

Classification distributions

If a classification file has been specified and the analysis performed was a single trait analysis, then, for a single trait analysis, the last 2N lines, N being the number of classification groups, in the model file will contain the distribution counts for the SNPs with respect to their classification are given as shown below:

```
Class 1
trait: 1 [1.9e+02, 0.17, 0.38, 41],[1, 1, 1, 1],233,41.26,0
Class 2
trait: 1 [2.1e+02, 0.91, 0.6, 52],[1, 1, 1, 1],267,53.03,0
```

Consider just Class 1:

The numbers in the first square brackets [1.9e+02,0.17,0.38,41], represents the number of SNPs falling into each of 4 distributions (1,2,3,4) during the post-burn-in analysis. The numbers in the 2nd square brackets [1, 1, 1, 1] represent the alpha distribution. The first number, 233, past the alpha distribution represent the number of SNPs assigned to that class minus the number of monomorphic SNPs in that class. The next number, 41.26, is the sum of the SNPs falling into distributions 2, 3 and 4. The zero at the end is the sum of values in the alpha distributions minus the number of distributions. This will usually be zero unless SNPs have been dropped during analysis.

The Poly file

The suffix '_Poly' is appended to the output prefix<id>. This is an optional file, which will only be produced if a pedigree or GRM file is specified in the parameter file or via the command line. It will contain 2 or more columns, one column for each trait analyzed. These columns contain the pedigree polygenetic effects for each record analyzed (Table 13)

Table 13: Example Poly file output

JERNZLM000000064048	-303.87
JERNZLM000000064076	-305.401
JERGBRM000000050200	-466.496
JERAUSM000A00000720	-508.295
JERAUSM000A00000734	-18.0926
JERAUSM000A00000747	261.63
JERAUSM000A00000768	-134.639

The Log file

The suffix '_log' is appended to the output prefix<id>. This is an optional file, which will only be produced if the command line option **-L** is used (see Table 1). It will contain run time information about the results being generated; such as parameter settings, the random number seed used, the variances, how long the bayesR3 will run etc. An example of the runtime output is given in the Appendix.

The SNPtrace file

The suffix '_SNPtrace' is appended to the output prefix<id>. This is an optional file, which will only be produced if output detail or **-d** command line option is set to 2. This will produce a large file, with the number of columns equal to the number of SNPs in the genotype file by the number of rows equal to the number of iterations minus the burn in iterations. This file will be written to disk as a compressed **gzip** file. The information in this file is useful to see if the SNP effects have converged over the burn in period.

The yhat file

The suffix '_yhat' is appended to the output prefix<id>. This file gives the predicated trait values, it will contain a minimum of two columns, the first column will give the associated IDs, this will be followed by columns of yhat values, one column for each trait analyzed. The order of the yhat columns, will match the order of traits given in the parameter file, or given in the **-tc** command line option (see Table 14).

Table 14: Example of a yhat file.

JERNZLM000000064048	-580.773
JERNZLM000000064076	-582.304
JERGBRM000000050200	-743.399
JERAUSM000A00000720	-785.198
JERAUSM000A00000734	-294.995
JERAUSM000A00000747	-15.2726
JERAUSM000A00000768	-411.541
JERAUSM000A00001711	-688.619

PredictBayesR3

PredictBayesR3 is used to do 3 things: validations, predictions and correlations and it takes a minimum of one command line argument, **<parameter_file>**:

```
%> predictBayesR3 <parameter_file>
```

It is designed to run using the same input parameters as passed to BayesR3. either by a parameter file or on the command line, but after the associated BayesR3 run. If the input and output files specifications use absolute addresses

than predictBayesR3 can be run from any directory. Otherwise, it should be run from the same directory BayesR3 was run from.

If the parameter **-L** is set to 1 then predictBayesR3's output, which gets printed to standard out (the screen), will get appended to the associated log file generated by BayesR3, if no associated log file was created by BayesR3 then predictBayesR3 will create one.

If the **-ppath** command is used to specify a directory path to be used by predictBayesR3, that by default is set to **NA**, will store all its output in this directory. If the directory contains a Log file, and **-L** command is set to 1, then predictBayesR3 standard output will be appended to this file, otherwise if needed a log file will be created. This effectively gives predictBayesR3 its own log file.

At this stage predictBayesR3 doesn't use the polygenetic effects that may have been produced by bayesR3.

Validation:

A validation is when the input parameters define a reference file, or a validation file, **phenotype** file and **genotype** file. When a validation file is used then all records specified in this file are included in the analysis. When only a reference file is used then all records in the phenotype file **not** included in the reference set are used to determine the model's accuracy. The output of a validation will be a <filename>_Valid_yhat text file, which will be placed in the output directory defined in the parameter file, and a **R^2** value and a **slope** value will go to the screen or log file.

Correlation:

A correlation analysis is performed when the input parameters **does not** define a reference, or a validation, file but defines only a **phenotype** file and a **genotype** file. Then all the records in the **genotype** that match records in the **phenotype** files are used to perform a correlation between the modelled phenotypes to the actual phenotypes given in the phenotype file. The output of a correlation will be a <filename>_Valid_yhat text file, which will be placed in the output directory defined in the parameter file, and a **R^2** value and a **slope** value will go to the screen or log file.

A correlation is generally used to make predictions using records that aren't defined in the training phenotype file. This could be the case where you have trained on a certain breed/strain of animals or plants and want to see if you can predict into a different breed or strain. However, the genotype file supplied must contain entries for the entries supplied in the "new phenotype" file. Example usage:

```
predictBayesR3.exe -phef new_phenotypes -valid NA -ref NA -fixf NA
```

In the example above the fixed effects file is also turned off. This is only required if the new phenotype file doesn't have entries in the fixed effect file.

Prediction:

A prediction is performed when the input parameters **does not** define a **phenotype** file but defines a **genotype** file. Here the **genotype** file is used to fit a set of predicted phenotypes. The output of a prediction will only be: <filename>_Valid_yhat text file.

If a reference file is supplied, then the prediction is performed using the genotypes from records **not defined** in the reference file.

If a validation file is supplied all genotypes specified in the validation file are used for the predictions.

It is an error to define a fixed effect file without defining a phenotype file.

Appendix

Common errors encountered:

- **Error: number of SNP rows in classification file NOT equal to number of SNP columns in file**
 - Often bayesR3 or predictBayesR3 reports a miss match between the number of records in one input file with another. While this can be the case, sometimes it's indicating there are duplicate record ID's in one or more of the input files.
- **Other common causes of problems:**
 - The genotype or other input file not being a space separated file.
- **Invalid parameter file**
 - Check for missing or misspelt parameters in the parameter file.

Example run time output

Checking validity of run parameters

Command Line entered:

```
C:\Users\vic5sx7\Dropbox\Projects\GWS Jersey\gaussSeidel\x64\Release\bayesR3.exe  
..\..\jerseyBull_paramNew.txt
```

Read reference file: ../../jerBull_validation.txt

Entries = 500

Reading phenotype(s) from ../../jerBull_pcaNANs_FMP_BayesR.txt

Time to load phenoTypes: 0.0043883(sec)
 phenotype (rows,cols) = 500,9

Selecting columns and weights
 Number of zero weights per column: 0
 phenotype (rows,cols) = 500,1

Reading genotype(s) from ../../jerBull_genotype_BayesR.txt.gz
 Time to load genotypes: 3.47548(sec)
 genotype (rows,cols) = 500,37008

Number of monomorphic SNPs: 4130
 Genotypes; min = -13.4096, max = 13.4096
 running chain: 0
 Running
 bayesR_N2: BUILD: Dec 11 2019 14:12:03

Time Per Iteration = 0.00750657 Expected Duration = 375.329 (seconds)
 Days 0 hours 0 minutes 6 seconds 15

Chain	Iteration	Varg	Vare	Vara	snpVar	Mean	Time
1	1	0	78278.4	88000.1	101508	1.02871	Wed Dec 11 14:12:30 2019
1	2401	0	7374.83	88000.1	116771	-0.893517	Wed Dec 11 14:12:47 2019
1	4801	0	14794.6	88000.1	110206	-2.45386	Wed Dec 11 14:13:04 2019
1	7201	0	14612.6	88000.1	108498	8.56514	Wed Dec 11 14:13:23 2019
1	9601	0	11755.1	88000.1	123471	3.60719	Wed Dec 11 14:13:42 2019
1	12001	0	17794.1	88000.1	109528	5.84121	Wed Dec 11 14:14:02 2019
1	14401	0	11713	88000.1	115080	-2.61404	Wed Dec 11 14:14:21 2019
1	16801	0	4639.15	88000.1	119877	0.116128	Wed Dec 11 14:14:41 2019
1	19201	0	13412.7	88000.1	109173	4.15075	Wed Dec 11 14:15:00 2019
1	21601	0	642.073	88000.1	133847	3.21832	Wed Dec 11 14:15:21 2019
1	24001	0	7803.09	88000.1	123434	0.508404	Wed Dec 11 14:15:42 2019
1	26401	0	12904.2	88000.1	123648	1.03653	Wed Dec 11 14:16:04 2019
1	28801	0	5294.08	88000.1	129151	-0.954723	Wed Dec 11 14:16:25 2019
1	31201	0	1211.83	88000.1	119902	4.87435	Wed Dec 11 14:16:46 2019
1	33601	0	4555.07	88000.1	129209	-0.302773	Wed Dec 11 14:17:06 2019
1	36001	0	6123.36	88000.1	128682	1.84593	Wed Dec 11 14:17:27 2019
1	38401	0	13900	88000.1	123284	9.01093	Wed Dec 11 14:17:47 2019
1	40801	0	8422.94	88000.1	131418	1.68696	Wed Dec 11 14:18:07 2019
1	43201	0	5522.85	88000.1	128284	2.09048	Wed Dec 11 14:18:30 2019
1	45601	0	13004.3	88000.1	110928	5.73308	Wed Dec 11 14:18:51 2019
1	48001	0	9300.18	88000.1	121387	3.86403	Wed Dec 11 14:19:11 2019

Model from chain: 1

Variance of the phenotypes = 1.6e+05
 Polygenic variance = 0
 Total SNP variance = 5.659e+04
 Error variance = 8424
 Total phenotypic variance = 6.501e+04

R^2 0.995
 Mean 2.371
 Vp 1.6e+05
 Va 8.803e+04
 Ve 8424
 Vg 0
 nMono 4130
 Associated 32878
 nSNPs 1.231e+04
 Nk1 2.057e+04
 Nk2 1.216e+04
 Nk3 141.3
 Nk4 5.896
 Pk1 0.6255
 Pk2 0.37
 Pk3 0.004298
 Pk4 0.0001793
 Vk1 0
 Vk2 4.903e+04
 Vk3 5598
 Vk4 1956

Writing: ../../keep/testingTc12_Effects.csv

Writing: ../../keep/testingTc12_Model.csv
Writing: ../../keep/testingTc12_yhat.csv

Example slurm script

The following gives an example of a simple slurm script.

The memory required for the SNP analysis can be determined from:

$$mem = 4.2 * (\#SNPS * (\#R + BS) + \#F * \#R + Ped + 2 * \#T * \#R)$$

Where $\#R$ is the number of records; $\#SNPS$ is the number of SNPS, $\#F$ is the number of fixed effects and Ped represents the elements required if a pedigree is included in the analysis where $Ped = \#R^2$; BS is the block size and $\#T$ represents the number of traits which gets double to account for the weights associated with each trait. Weights are used even if the user doesn't specify any weight columns. In this case the weights are usually just 1 or 0, which account for any missing phenotypes.

The time is usually estimated from knowing the time required to analyze other similar sized data sets, or by running a timing analysis.

Chains and number of cores/threads (cpus-per-task):

The number of cores/threads should be at least 8, and for a multi-chain analysis no less than **max(4, #Chains)**. However, the number 8 is somewhat arbitrary, as BayesR3 only needs 1 core per chain once it starts solving the genomic equations. Also, during startup BayesR3 processes several tasks in parallel if possible, such as loading genotype files, building blocks etc. For example, if you split the genotype file into 100 parts, then you might want to use 100 cores. If you want to run 10 chains, then set cores to 10 etc.

```

#!/bin/bash

# Copy/paste this job script into a text file and submit with the command
# update parameter file name and other task specific commands
# sbatch thefilename

#SBATCH --mem=50GB
#SBATCH --time=72:00:00 # 3 days walltime limit (HH:MM:SS)
#SBATCH --nodes=1 # number of nodes
#SBATCH --cpus-per-task=8 # 8 processor core(s) per node
#SBATCH --job-name="Phuong3X"
#SBATCH --mail-user=ed.breen@agriculture.vic.gov.au # email address
#SBATCH --mail-type=END
#SBATCH --output="bayesR3%j.out" # job standard output file (%j replaced by job id)

# LOAD MODULES, INSERT CODE, AND RUN YOUR PROGRAMS HERE

module load GCCcore

/group/bin/bayesR3 paramT.txt _180000_ -L

/usr/local/bin/showJobStats.scr

```

Multi-trait PCA values

The multi-trait analysis performed by bayesR3 assumes that traits are uncorrelated and if you can't assume the traits are independent, then one way to ensure this is to run a PCA analysis on the traits. The following R function `getPCAvalues` can be used to generate a PCA file suitable for use as a phenotype file in a multi-trait bayesR3 analysis using a given bayesR phenotype file.

The resulting PCA file output is written to same directory that contains the original phenotype file, but with the file extension `".pca.txt"`. Therefore, the input file name is expected to contain its directory path. The first output column in the PCA file will contain the record IDs specified in the original phenotype file, and the following columns will contain the PCA values from 1 to N, where N is the number of PCA components that explain for example 90% of the total variances in your data. The actual percentage is user determined and is set using the `thresh` parameter, and its default value is 0.9:

```
getPCAvalues = function(infile, thresh=0.9, insep='\t')
{
  # infile is the name of a bayesR3 multi-trait phenotype file
  #       assumes that infile also contains the directory path to the phenotype file
  # insep specifies the field separator in the input file infile
  # thresh defines the total variance to be explained by the PCA components
  #       writes the PCA components to a file with the file suffix ".pca.txt" to the same directory that
  #       contains the original phenotype file.

  dir = dirname(infile)

  outfile = strsplit(basename(infile), "\\")[[1]][1]
  outfile = paste(dir, '/', outfile, ".pca.txt", sep="")

  data = read.csv(infile, sep=insep)

  pca = prcomp(data[,-1], center = T, scale. =T)
  eig = pca$sdev^2

  y = eig/sum(eig)
  yc = cumsum(y)

  i = which(yc >=thresh)[1]

  df = data.frame(data[,1], round(pca$x[,1:i],4))
  write.table(x = df, file = outfile, quote = F, col.names = F, sep = ' ', row.names = F)
}
```

Change log

- Missing genotypes are now automatically detected and imputed. The genotype codes for missingness are a period (.), an asterix (*) or NA. Missing genotypes will be replaced by the mean genotype value for the given marker.
- Added new command switch **-ppath**, which if set, sets a directory path to be used by predictBayesR to write its results into and separately from bayesR3
- Fixed problem with bayesR3 and predictBayesR3 not determining if a file opened correctly
- predictBayesR3 can now also use a split prefix genotype file names.
- During genotype read, bayesR3 now checks for duplicated entries (IDs)
- Sped up the setting up of blocks in a multi-trait analysis.
- Added the new command **-parts** and if set to 1 the genotype file name is considered a prefix/ (regular expression), otherwise the genotype file name is considered to represent a single file. **Default value is 0.**
- Genotype file names are now be treated as prefix names, if the **-parts** switch is set to 1. When the genotype file is split into parts it can be read in parallel thereby speeding up the loading of a genotype file, which at times may take many hours.

To use a genotype prefix, pass bayesR3 via **-genf** the output prefix that was used to split the original genotype file into N parts as used by split:

```
○ zcat subSet4e4.txt.gz | split - -l10000 subParts4e4
```

Note:

- **Only split non-zipped genotype files.**
- **The prefix should be unique with respect to split files.**

For a genotype file containing 65,000 records/lines the above split command will generate the following 7 files:

```
subParts4e4aa subParts4e4ab subParts4e4ac subParts4e4ad  
subParts4e4ae subParts4e4af subParts4e4ag
```

Each file will contain 10,000 records/lines except for the last file (subParts4e4ag), which will contain only 5,000 records/lines.

- Genotype values outside the range 0 to 2 will now cause bayesR3 to throw an error.
- Pedigree files are now allowed to have more than 3 columns, but only the first 3 columns are interpreted as animal ID, sire ID and dam ID, and in that order.

Update to Saturday, Tuesday, September 7, 2021

- Fixed several bugs
- Added command option plusAlpha
- Reimplemented the dropping of SNPs. Dropping of SNPs occur only during burn in. The **drop** parameter is used specify to bayesR3 to drop SNPs from analysis that are consistently not being included in the bayesR3 model. The **drop** parameter takes 1 or 2 comma separated values. Default value is NA, which is used to specify no dropping.

For example:

-drop 100

Specifies that during burn in, a SNP will be dropped from analysis if in the last 100 iterations it was not included in the model. In this case dropping can only occur on and after the 100th iteration.

-drop 100,200

Specifies that during burn in, a SNP will be dropped from analysis if in the previous 100 iterations it was not included in the model. In this case dropping starts at the 200th iteration.

-drop NA

Specifies no dropping.

- predictBayesR3: fixed bug/feature that the genotype and phenotype files need to have the same records when using a reference file. Now only the IDs in the phenotype file not included in the reference file need to exist in the genotype file therefore the genotype file can contain extra records not defined in the genotype file.
- Added a new bayesR3 command option “-n”, which is used to specify an **integer** number of inner-cycles, as opposed to the current method of setting inner-cycle number to equal the block size. Lowest value allowed is 1, if specified otherwise “-n NA” or “-n -1” turns the option off and it is off, without specification, by default.
 - Note: this only changes the number of inner cycles, and how the inner and outer cycle mix to maintain the same number of required iteration and burn-in values specified in a parameter file
- Fixed bug of bayesR3 crashing when certain parameters in the parameter were not defined or missing from the parameter file.

Update to Monday, January 25, 2021

- Corrected overflow problem for the reporting of the number of SNPs in class distributions.

Update to Wednesday, September 30, 2020

- Fixed issue with bayesR3 not computing the average inclusion probabilities (Q1 and Q2) in a multi-trait analysis across the chains. This is the result that ends up in the aveEffects file as specified in see Table 9.

Updates to Thursday, September 24, 2020 (release date)

- A “Common errors encountered” section has been added to bayesR3’s documentation pdf.
- Removed the prior previously added in the estimation of V_a . It was found on average to reduce prediction accuracy slightly.
- Added the ability for bayesR3 to incorporate family structure using a GRM (genomic relationship matrix) as described in its documentation pdf.
- A slight improvement in the average predication accuracy of bayesR3, was achieved by accumulating each SNP value directly after their sampling event during post-burn in. This is opposed to the previous strategy of collecting SNP effects during post-burn in at the end-of-each outer cycle.
- Fixed the error with the reported V_p value(s) in the log and model files, that was caused by dividing it wrongly by the number of post-burnin outer-cycles.
- Removed the artificial lower limit of 0.0002 imposed on the user’s specified minor allele frequency (MAF). It can now be any value greater than zero but less than or equal to 0.49.

Updates to Tuesday, September 1, 2020 (release date)

- Added a change log section at the end of the appendix section of the bayesR3 documentation
- Finally managed to fix the Linux problem of core dumping. Fingers crossed
- Added an extra output file when doing multi-chain analysis. In a multi-chain analysis, the SNP effects and their distribution proportions from each chain are collected averaged and stored with the other output files, but with the suffix **aveEffects**. The SNP effects reported in this file will be stored along with their chain variances and will be placed next to the effects in a column titled VB for a single trait analysis and for a multi trait analysis they are identified by VB.1,..., VB.n where n is the number of traits analyzed:

Freq	PIP1	PIP2	PIP3	PIP4	Beta	VB
0.667	0.172	0.015	0.108	0.705	1.910	1.045
0.979	0.304	0.026	0.100	0.570	1.401	0.552
0.794	0.459	0.038	0.048	0.455	1.017	0.890
0.839	0.609	0.051	0.103	0.237	0.547	0.560
0.372	0.580	0.051	0.151	0.218	0.533	0.491

- Improved the speed of inputting files such as fixed effects, genotype, and phenotype files. The speed improvement comes about not by faster file reading but by not having to reorder the rows in each input matrix to match up across the various inputs.
- In bayesR3, moved the reading of the fixed effects file, if passed, before the reading of the genotypes. This saves time when reporting errors to the user if there are problems, such as file formatting issues.
- Added a prior to the calculation of the SNP effects variance that gets reported as **Va**. This is only active if **DAV** is set to 1. The prior uses the proportions from the parameter file, or command line:

-pi 0.94,0.049,0.01,0.001

to estimate the number of expected SNPs that should fall into the model. This in turn makes bayesR3 sensitive to these initial values. Therefore, for genotypes of 600K or larger, consider using:

-pi 0.994,0.0055,0.00049,0.00001

- predictBayesR3 now responds to the log file command -L. If -L 1 is passed or set in the parameter file, predictBayesR3 will append its output results (R² and slope) to the log file created by bayesR3. If such a file doesn't exist, then it will create it according to the prefix and path values set in the parameter file or passed on the command line.
- Fixed the problem with bayesR3 and predictBayesR3 of ignoring user specified seed values in the parameter file or passed on the command line. As only a single set of seed values can be specified then for a multi-chain analysis, the specified seed values will be divided into equal sized (if possible) seed sets for each chain. Therefore, in a multi chain analysis the number of seeds must be equal or greater than the number of chains specified.
- Added command input validation. Previously, bayesR3/predictBayesR3 command and command values from parameter files or specified on the command line were accepted without any checking. Now they are checked, and any entered command is not in the set of valid commands an error is reported and the program stops.
- Extended the accuracy of calculating the association probabilities in a multi-trait bayesR3 analysis. Previously, multi-trait analysis was limited to around at most 200 traits per analysis. Now bayesR3 can handle correctly at least 4608 traits (that is 23-fold improvement in the power of a multi-trait analysis).
- Added the use of validation sets (-valid <filename>) in conjunction with the use of a reference (-ref <filename>) set.
 - A validation set is only used by predictBayesR3. The validation set defines a set of records that are included into predictBayesR3's prediction analysis, as opposed to just using a reference set, which specifies which records not to include in the prediction. Note if both reference and validation files are specified in a parameter file

then bayesR3 uses the reference set, while predictBayesR3 uses only the validation set. Either or both can be set to NA.

- Validation and reference files can now contain multiple columns. However, only the first column is used and is expected to contain the record IDs.
- Fixed bug in the computing of fixed effects
- Fixed minor bug with computing the Dirichlet probabilities

References:

VanRaden, P. M. (2008). "Efficient methods to compute genomic predictions." J Dairy Sci **91**(11): 4414-4423.